ТЕХНОЛОГИИ ХРАНЕНИЯ БОЛЬШИХ ДАННЫХ

Парадигма MapReduce, Hadoop, HDFS

Hadoop



- **Hadoop - это программная среда с открытым исходным кодом, используемая для распределенного хранения и обработки наборов данных больших данных с использованием модели программирования MapReduce.**
- **Он состоит из компьютерных кластеров.**
- **Все модули в Hadoop разработаны с фундаментальным предположением, что аппаратные сбои являются обычным явлением и должны автоматически обрабатываться платформой.**

История развития фреймворка Hadoop

- Статья "Файловая система Google", опубликована в октябре 2003 года;
- Hadoop 0.1.0 был выпущен в апреле 2006 года;
- HBase и Pig были созданы в октябре 2007 года;
- В 2009 году Hadoop был использован для обработки петабайта данных;
- Выпуск Hadoop 1.0: ноябрь 2012 г .;
- Apache Spark был готов к 2014 году;
- Hadoop 3.0 был выпущен в 2017 году.
- Развитие Hadoop продолжается и сейчас.

Для чего нужен Hadoop?

- **Гибкость для хранения и обработки данных любого типа, будь то структурированные, полуструктурированные или неструктурированные. Он не ограничен одной схемой.**
- **Преимущество при обработке данных сложного характера. Его масштабируемая архитектура распределяет рабочие нагрузки по многим узлам.**
- **Еще одним дополнительным преимуществом является то, что его гибкая файловая система устраняет узкие места ETL.**
- **Экономно масштабируется, может использоваться на обычном оборудовании. Помимо этого его природа с открытым исходным кодом защищает от vendor lock.**

Концепции Hadoop

Масштабируемость для больших объемов данных:



23 дня

Сканирование 100 ТБ при 50 МБ / с на **1 узле**



33 минуты

Сканирование на кластере **1000 узлов**

Экономическая эффективность:

- ✓ Использование узлов потребительского класса
- ✓ Стандартное оборудование для сетей
- ✓ Автоматическое восстановление после сбоя
- ✓ Легко использовать

Концепции Hadoop

Дешевое оборудование часто дает сбои, особенно если у нас его много:

- среднее время до отказа для 1 узла = 3 года
- для 1000 узлов = 1 день

Решение: встроенная защита от сбоев

Стандартное межсоединение = низкая пропускная способность:

Решение: перемещение вычислений в данные

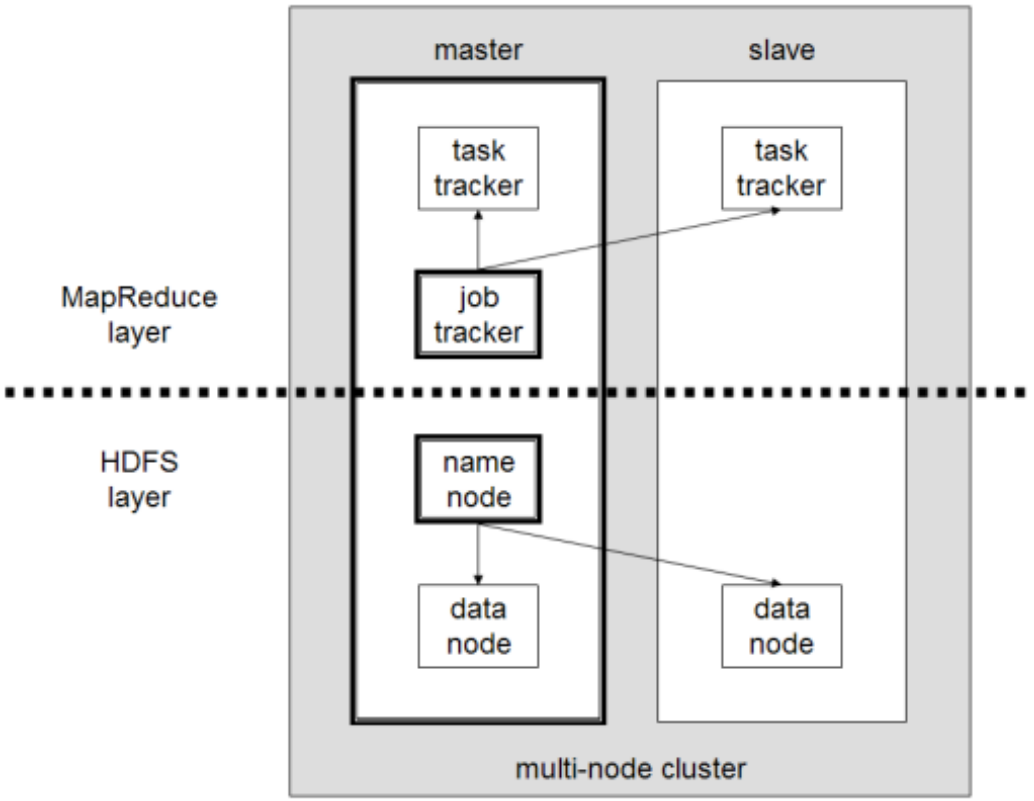
Программирование распределенных систем - сложная задача:

Решение: Предоставление программисту вычислительной среды, в которой он будет регистрировать только функции map и reduce

Концепции Hadoop

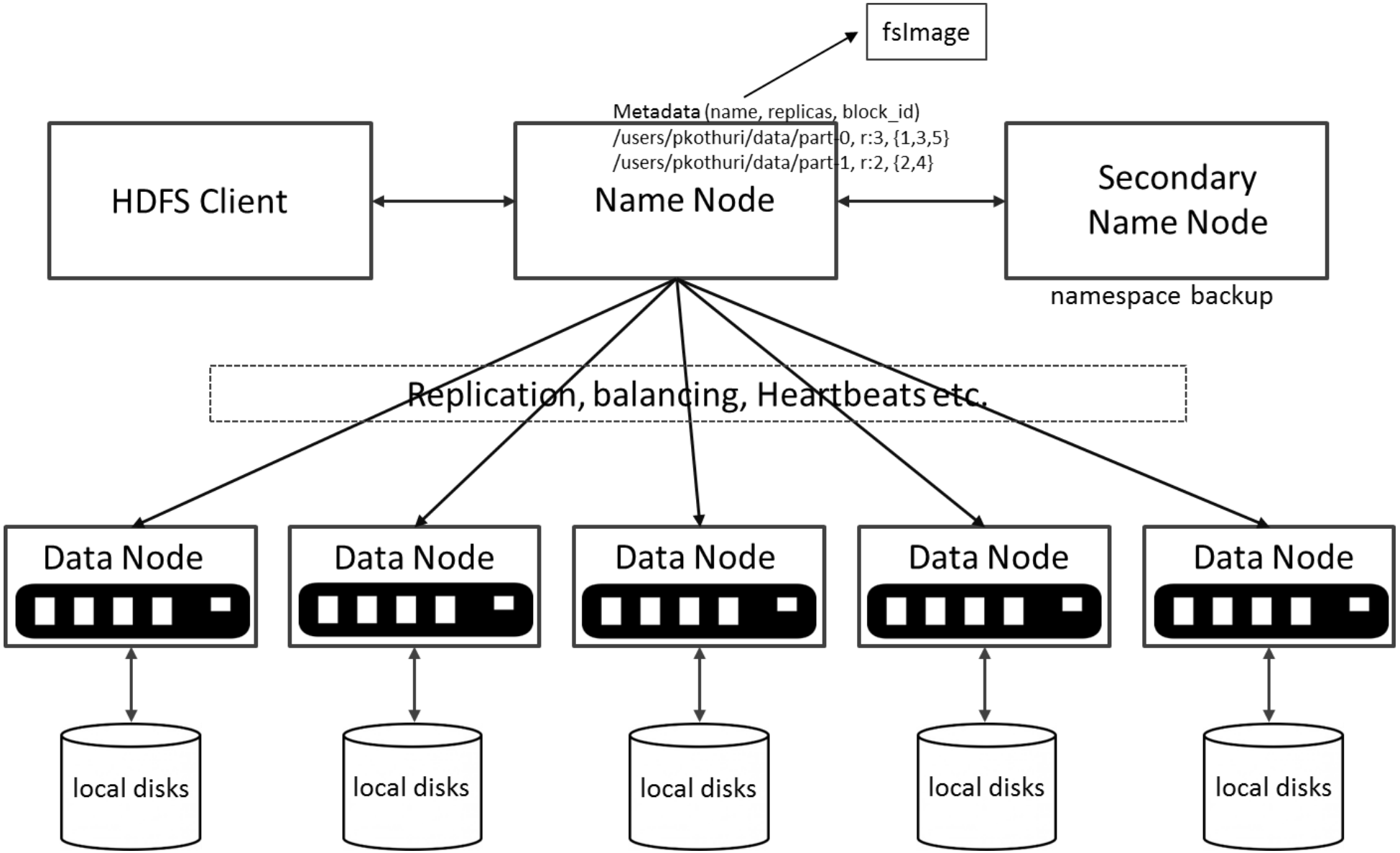
- **Автоматическое распараллеливание**
- **Отказоустойчивость**
- **Инструменты мониторинга состояния**
- **Понятная абстракция для программистов**
- **Программы MapReduce пишут на JAVA или на любом другом сценарном языке**
- **Hadoop написан на Java**

Архитектура Hadoop



- **Hadoop работает по системе master-slave. Есть главный узел и есть подчиненные узлы.**
- **Главный узел управляет, поддерживает и контролирует подчиненных, тогда как подчиненные выполняют вычислительные задачи.**
- **В архитектуре Hadoop Master должен развертываться на хорошем серверном оборудовании, а не на обычном оборудовании.**

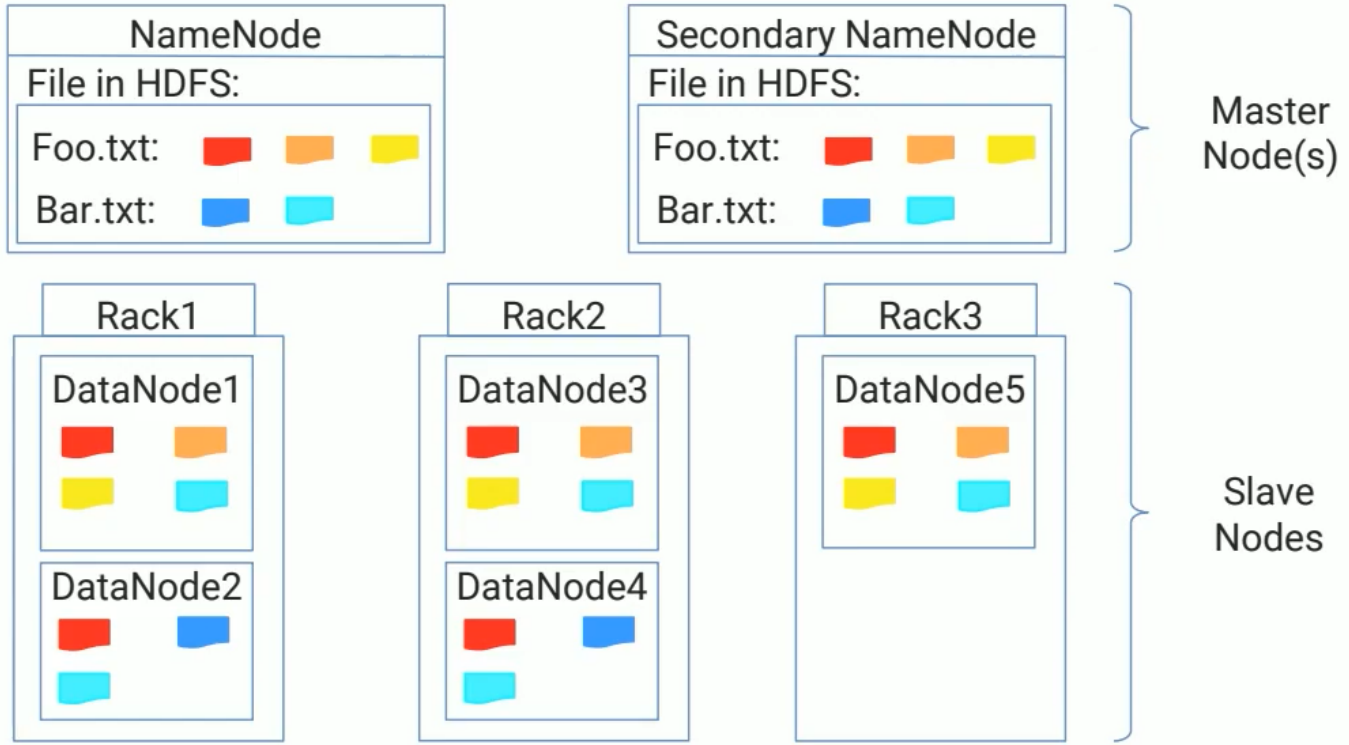
HDFS: структура



HDFS: структура

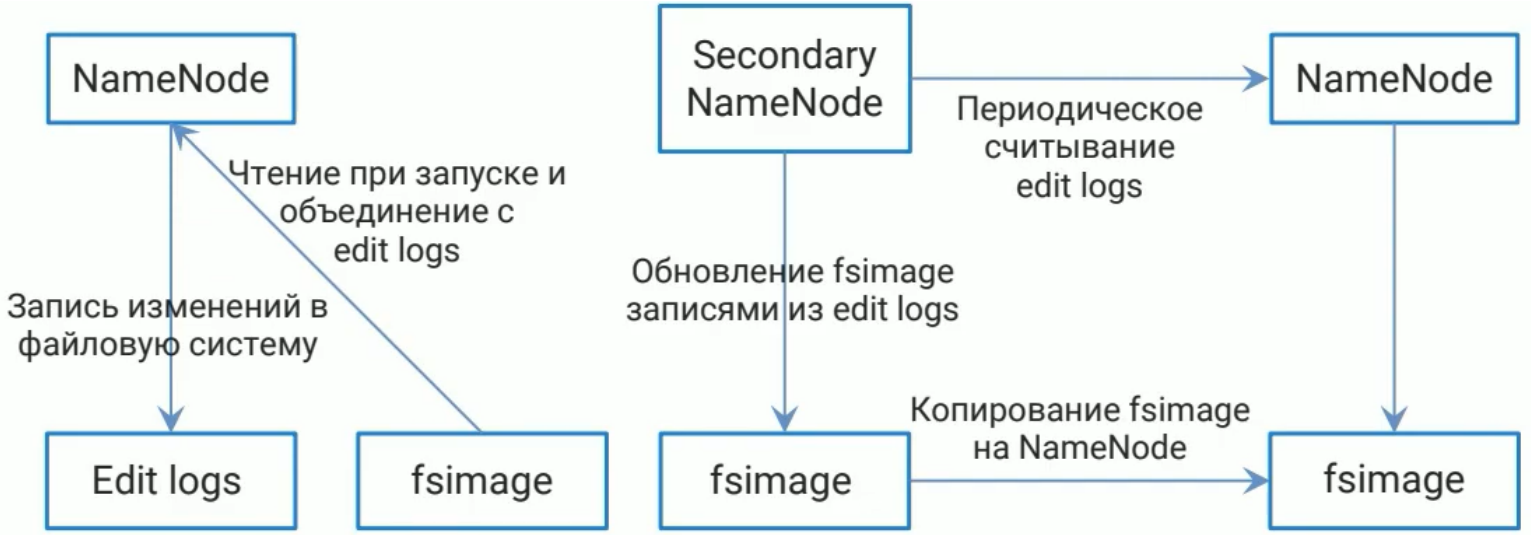
- **NameNode: сердце файловой системы HDFS, он поддерживает и управляет метаданными файловой системы. Например; какие блоки составляют файл и на каких DataNode эти блоки хранятся.**
- **DataNode: здесь HDFS хранит фактические данные, обычно этих узлов довольно много.**

Репликация файлов в HDFS



- Содержимое файла разбивается на блоки (по умолчанию 128 МБ; настраивается dfs.blocksize)
- Каждый блок реплицируется на несколько узлов данных (настраивается dfs.replication)
- NameNode отслеживает количество реплик блока
- NameNode создает новую реплику блока при сбое DataNode

NameNode



- Должен работать постоянно, иначе кластер становится недоступным
- Ведет учет изменений на диске для восстановления после сбоя:
  - fsimage - снимок файловой системы
  - edit log - последовательность изменений, внесенных в файловую систему после запуска NameNode
  - Secondary NameNode - отдельный демон, выполняет некоторые служебные задачи для NameNode

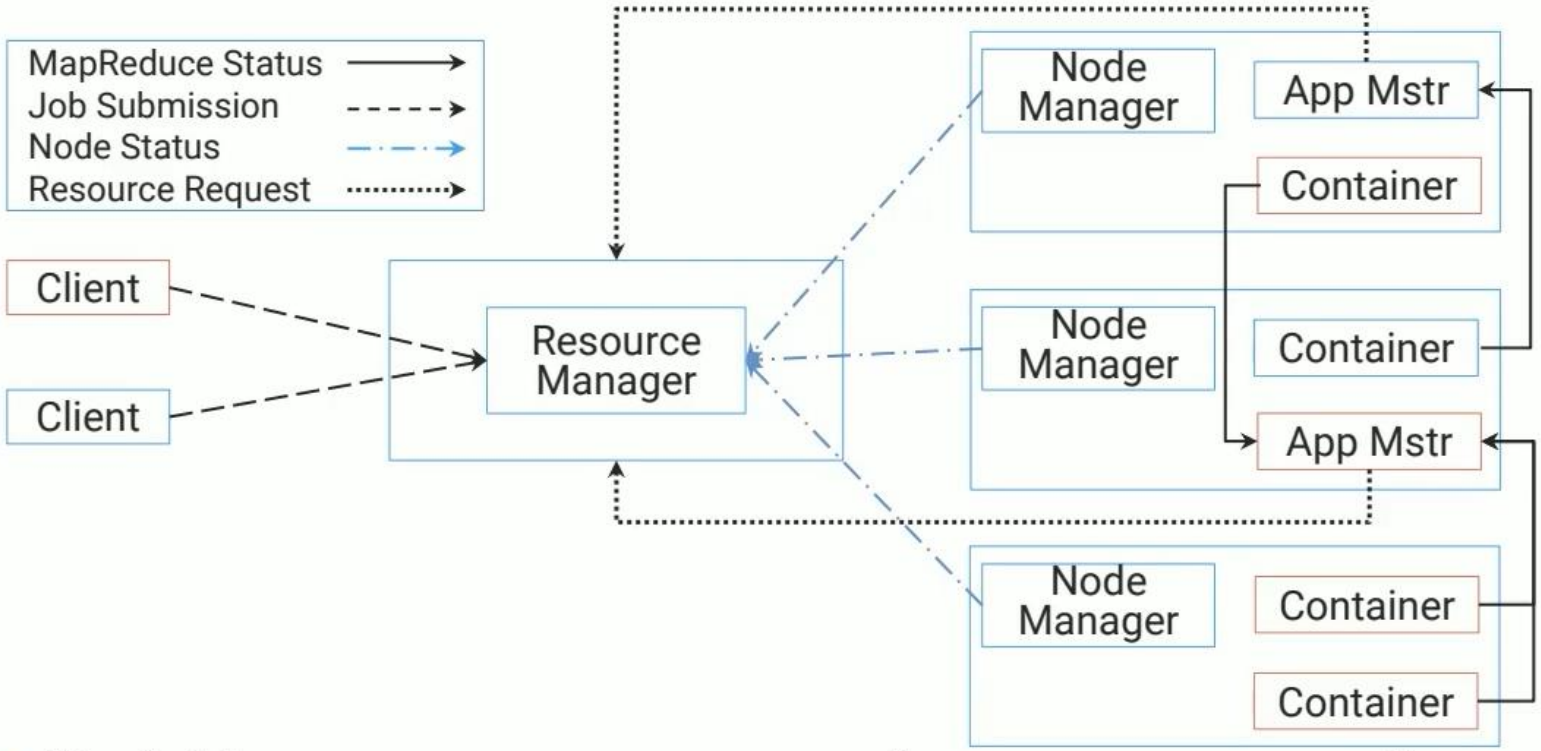
Добавление и выведение из эксплуатации DataNode

- Файл dfs.include используется для добавления новых узлов данных в кластер
- Запустить hdfs dfsadmin – refreshNodes, чтобы обновить топологию кластера после добавления нового DataNode и перебалансировать реплики
- dfs.hosts.exclude содержит узлы для отключения
- Повторить hdfs dfsadmin – refreshNodes
  - Веб-интерфейс NameNode на странице «Узлы данных» показывает статус «Выполняется вывод из эксплуатации»
  - Если состояние всех узлов данных «сняты с эксплуатации» значит все блоки реплицированы
- Отключить списанные узлы
- Удалить отключенные узлы данных из файла dfs.include и повторно запустить hdfs dfsadmin – refreshNodes

Yet Another Resource Negotiator

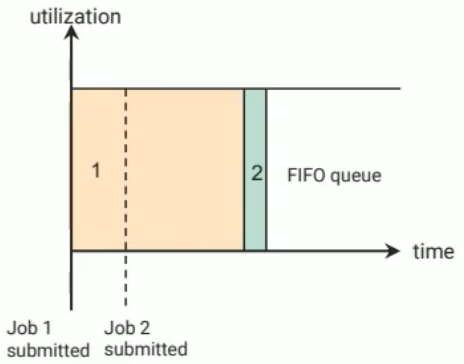
- **Технология управления ресурсами с открытым исходным кодом, которая развернута в кластере Hadoop.**
- **YARN стремится эффективно распределять ресурсы между различными приложениями.**
- **Он запускает два процесса, которые выполняют две разные задачи: отслеживание заданий и мониторинг прогресса.**
- **Эти два демона называются resource manager и application master / node manager соответственно.**
- **Диспетчер ресурсов распределяет ресурсы между различными приложениями, а мастер приложений контролирует выполнение процессов.**

Компоненты YARN

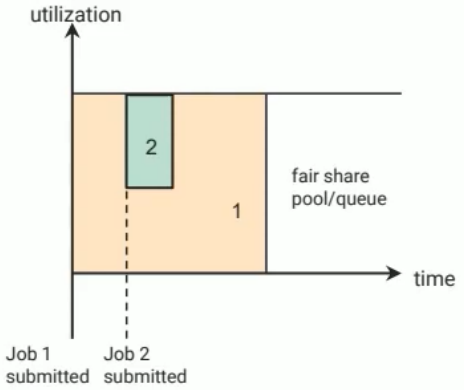


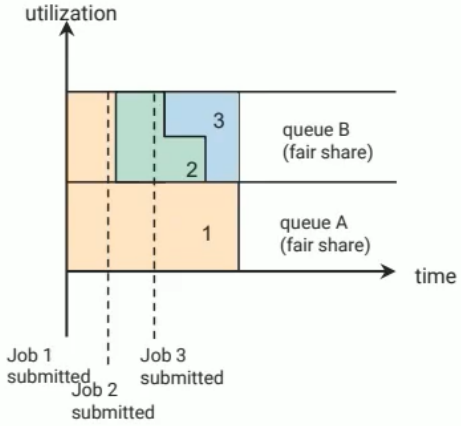
- NodeManager запускает контейнеры приложений на узлах
- ApplicationMaster - управляет жизненным циклом приложений, согласовывает ресурсы с ResourceManager, выполняет мониторинг с помощью N

Виды планировщиков YARN

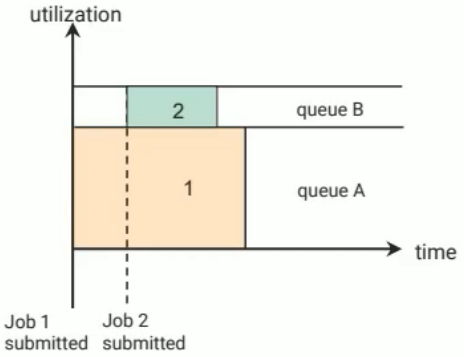


FIFO scheduler





Fair scheduler



Capacity scheduler

По умолчанию Hadoop использует планировщик FIFO, но в промышленных задачах чаще используют fair (Cloudera CDH) или capacity (Hortonworks HDP) планировщики.

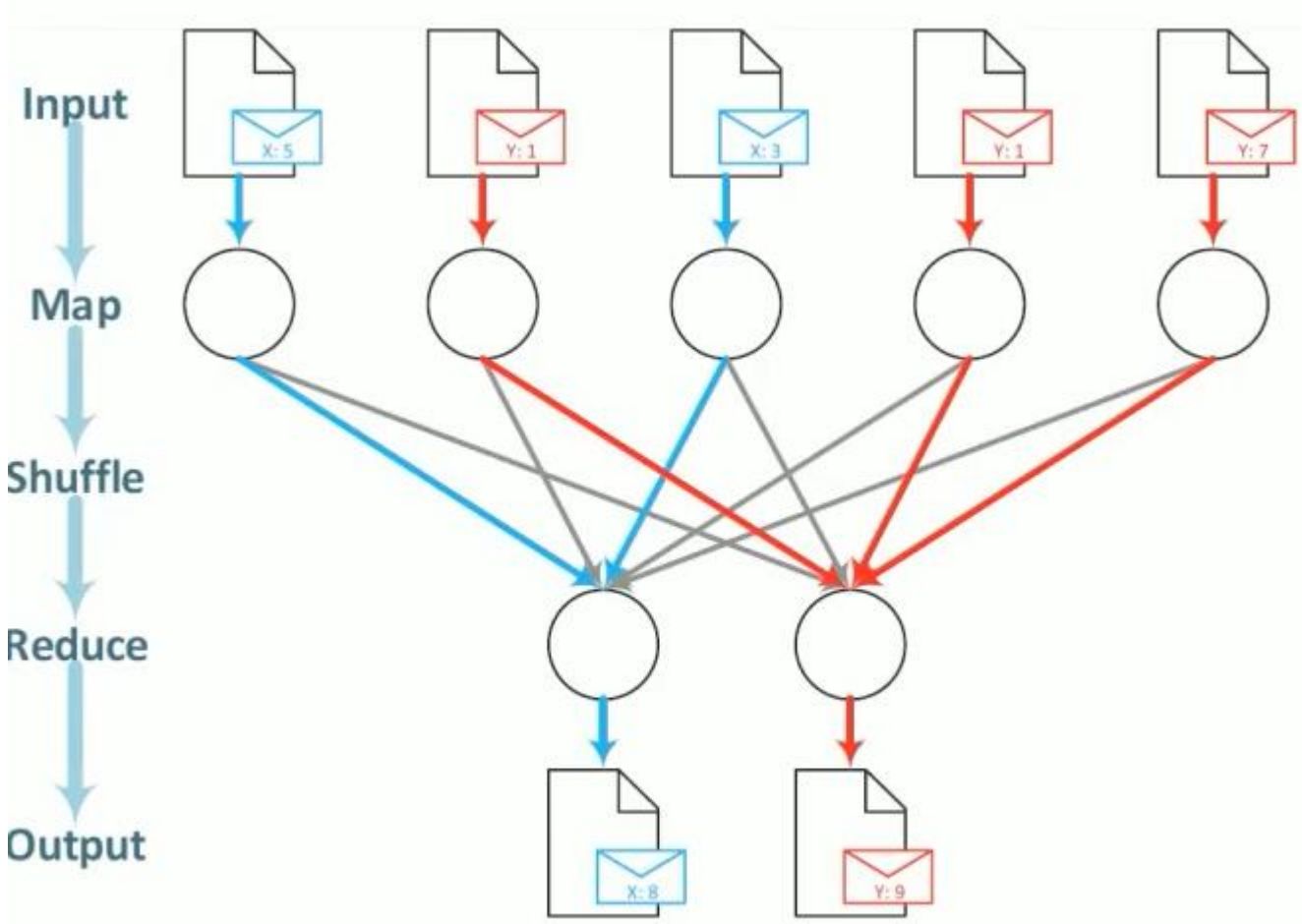
Для чего нужен YARN

- **Гибкость - включает другие специализированные модели обработки данных, помимо MapReduce (пакетные), такие как интерактивная и потоковая. Благодаря этой функции YARN, другие приложения также могут запускаться вместе с программами Map Reduce в Hadoop.**
- **Эффективность. Поскольку многие приложения работают в одном кластере, эффективность Hadoop увеличивается без особого влияния на качество обслуживания.**
- **Shared - обеспечивает стабильную, надежную, безопасную основу и разделяемые операционные сервисы для нескольких рабочих нагрузок. Дополнительные модели программирования, такие как обработка графов и итеративное моделирование, теперь возможны для обработки данных с помощью YARN.**

Hadoop Map Reduce

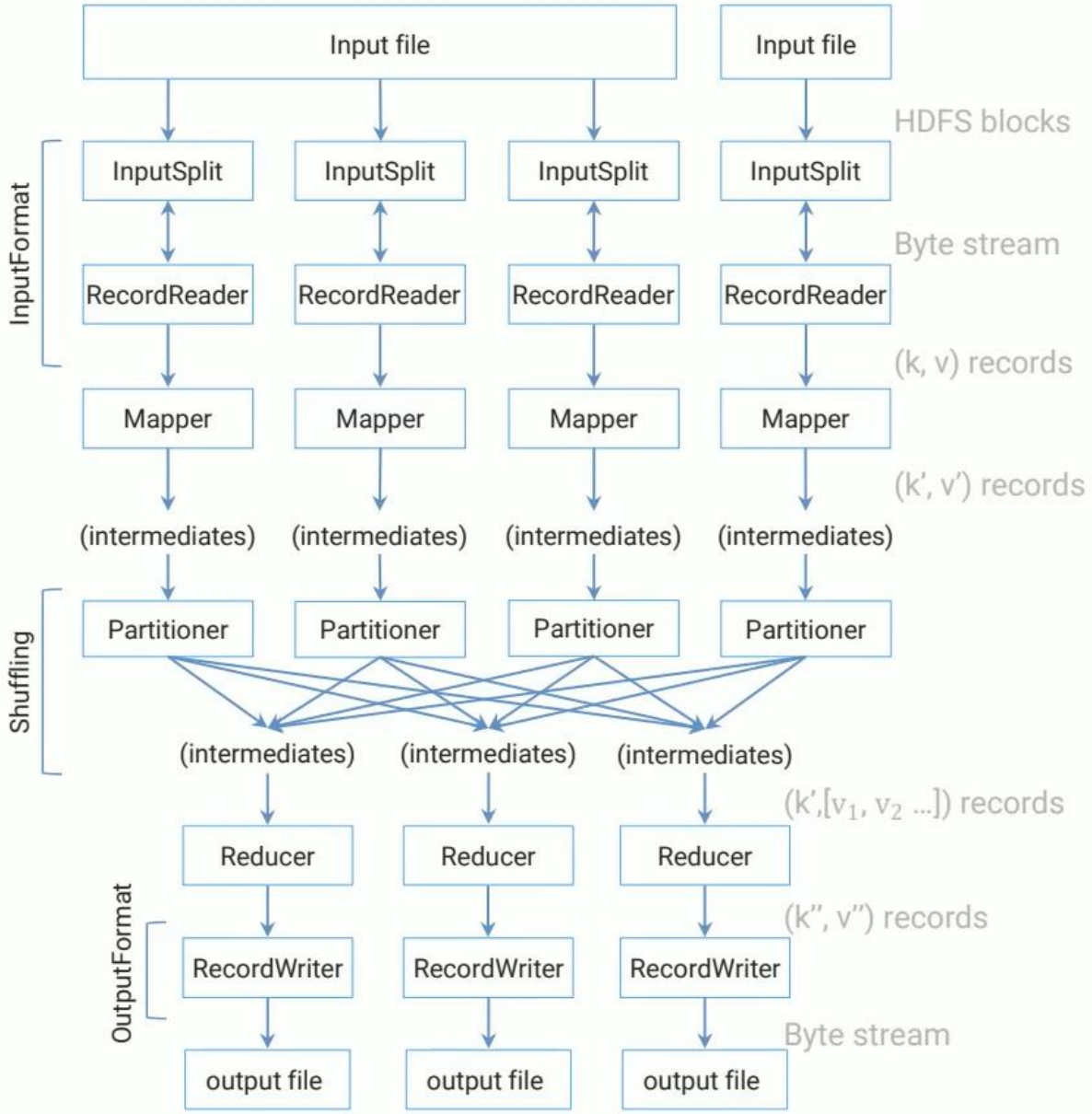
- **Простота - задания MapReduce легко выполнять. Приложения могут быть написаны на любом языке, например, Java, C ++ и Python.**
- **Масштабируемость - MapReduce может обрабатывать петабайты данных.**
- **Скорость - с помощью параллельной обработки, решение задач, на которые обычно уходят дни, выполняется за часы или минуты с помощью MapReduce.**
- **Отказоустойчивость - MapReduce устойчива к сбоям. Если одна копия данных недоступна, на другой машине будет копия данных, которую можно использовать для решения подзадачи.**

Как работает MapReduce



- **Задача Map**
  - **Обрабатывать блоки входного файла**
  - **Произвести пары ключ-значение**
- **Задача Reduce**
  - **Взять пары ключ-значение, сортировка по ключам**
  - **Объединить**
  - **Вывести окончательные результаты**

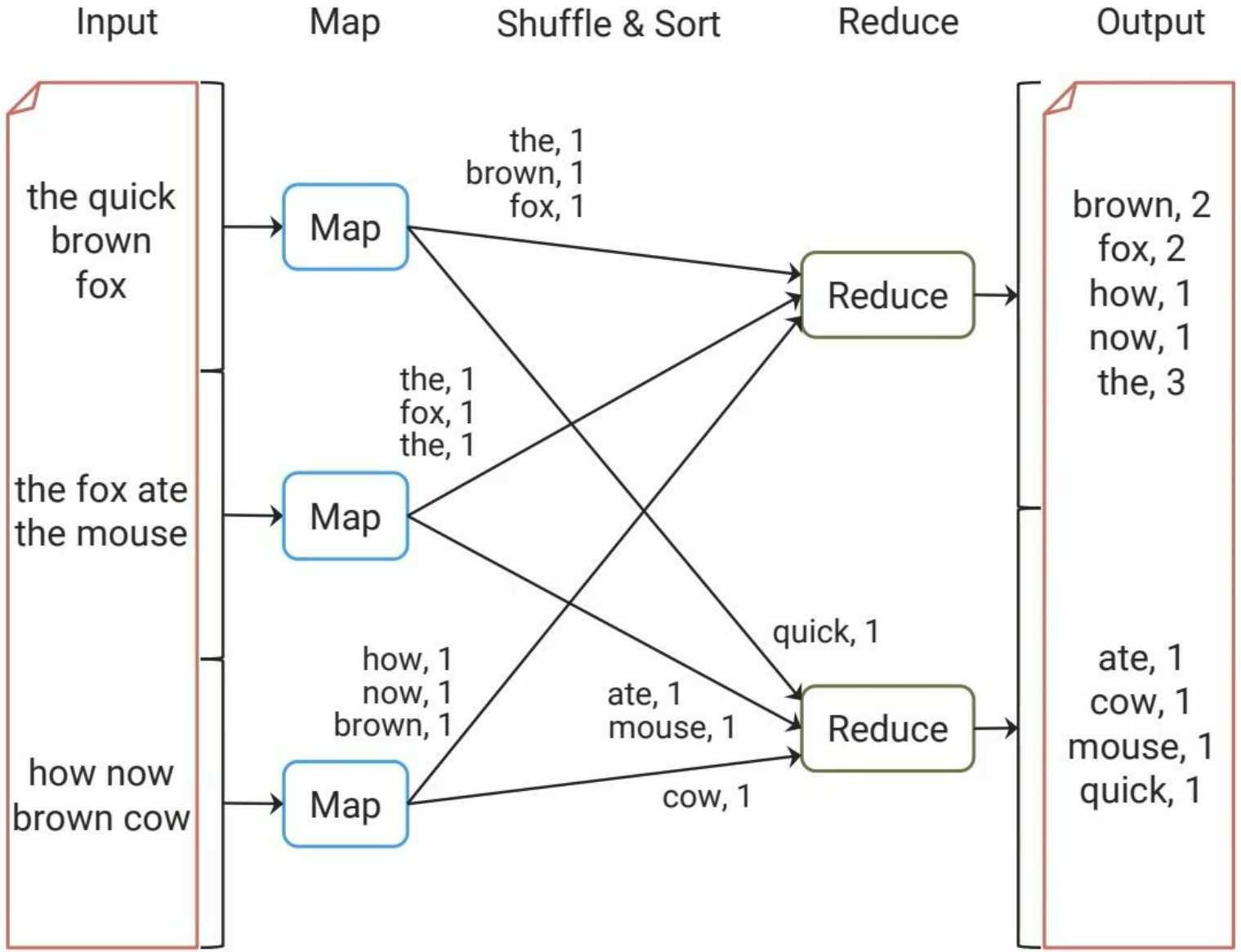
Как работает MapReduce



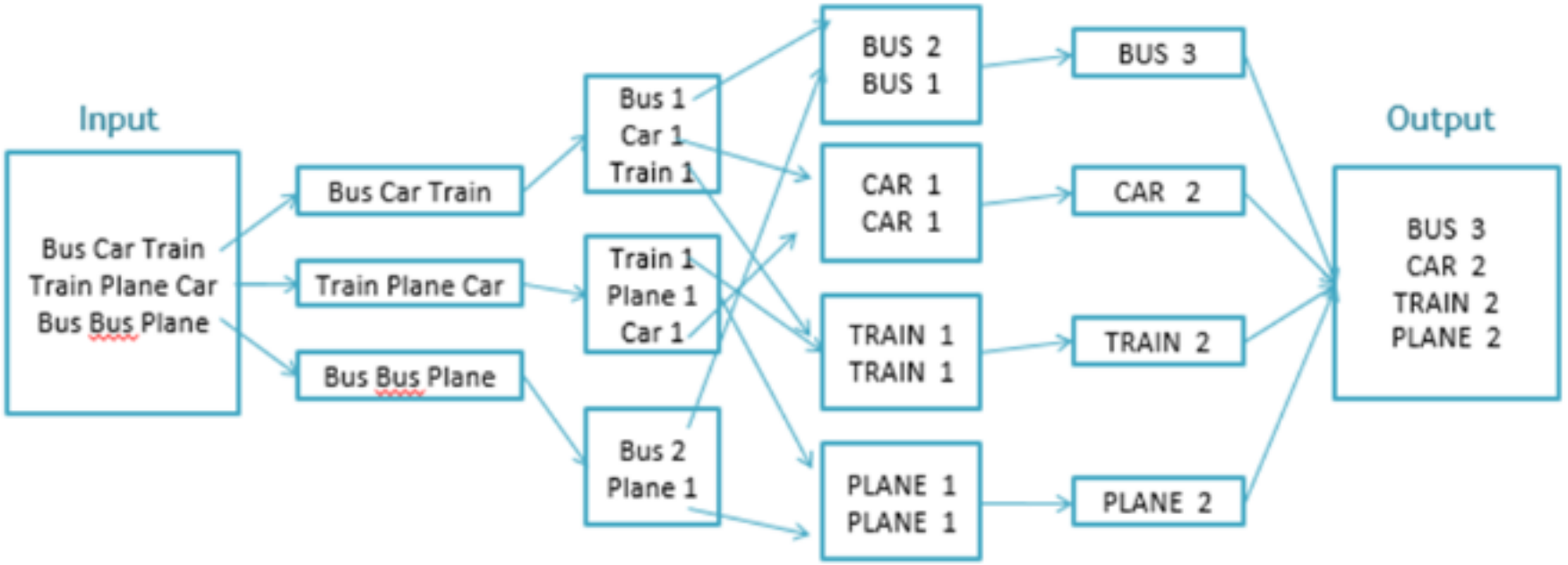
Word Count на Python

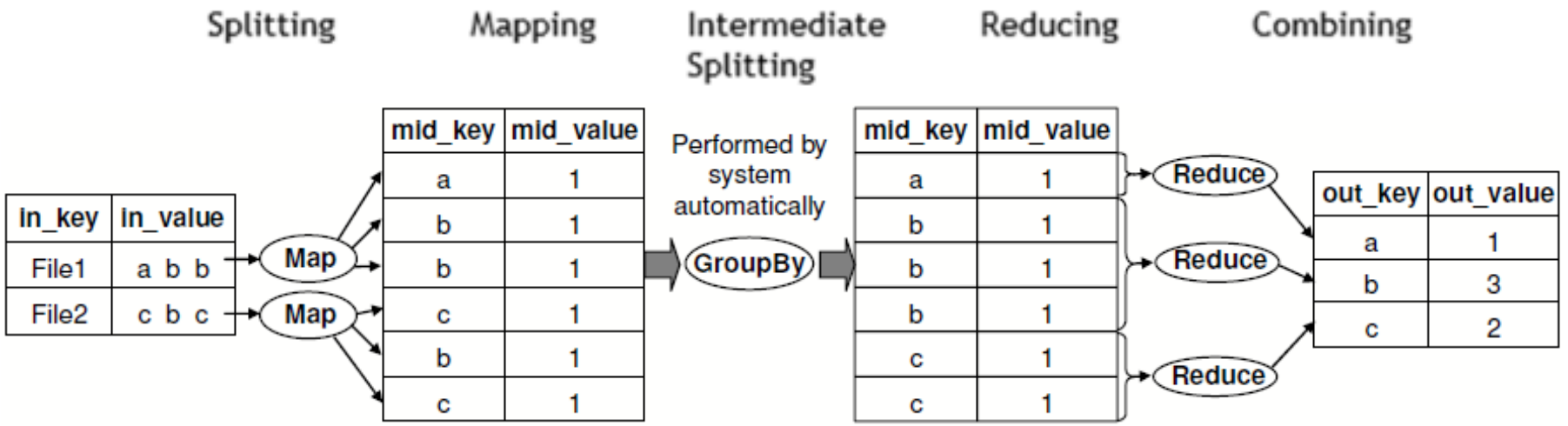
```
def mapper(line):
        foreach word in line.split():
            output(word,1)

def reducer(key,values):
        output(key, sum(values))
```



Word Count


Input
Bus Car Train
Train Plane Car
Bus Bus Plane
Bus Car Train
Train Plane Car
Bus Bus Plane
Bus 1
Car 1
Train 1
Train 1
Plane 1
Car 1
Bus 2
Plane 1
BUS 2
BUS 1
CAR 1
CAR 1
TRAIN 1
TRAIN 1
PLANE 1
PLANE 1
BUS 3
CAR 2
TRAIN 2
PLANE 2
Output
BUS 3
CAR 2
TRAIN 2
PLANE 2


Splitting
Mapping
Intermediate Splitting
Reducing
Combining
in_key
in_value
File1
a b b
File2
c b c
Map
Map
mid_key
mid_value
a 1
b 1
b 1
c 1
b 1
c 1
Performed by system automatically
GroupBy
mid_key
mid_value
a 1
b 1
b 1
b 1
c 1
c 1
Reduce
Reduce
Reduce
out_key
out_value
a 1
b 3
c 2

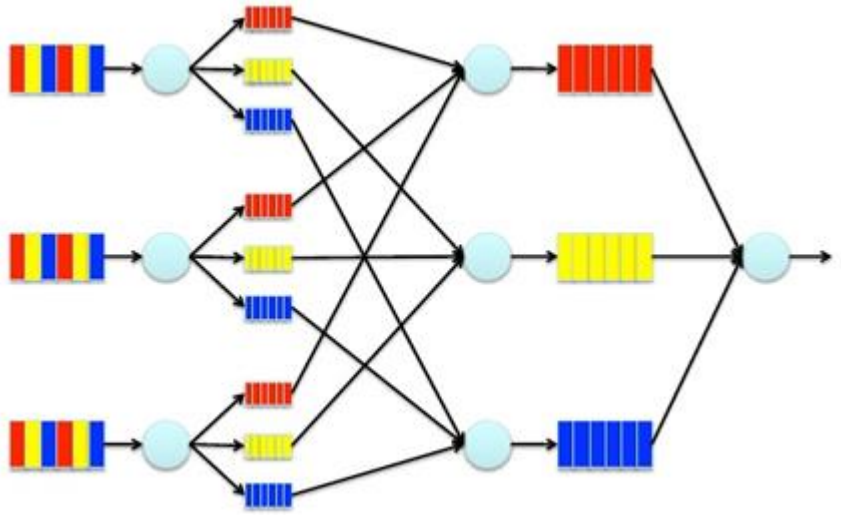
Детали MapReduce

- **Все запуски функции map работают независимо и могут работать параллельно, в том числе на разных машинах кластера.**

- **Все запуски функции reduce работают независимо и могут работать параллельно, в том числе на разных машинах кластера.**

- **Shuffle внутри себя представляет параллельную сортировку, поэтому также может работать на разных машинах кластера. Пункты 1-3 позволяют выполнить принцип горизонтальной масштабируемости.**

- **Функция map, как правило, применяется на той же машине, на которой хранятся данные – это позволяет снизить передачу данных по сети (принцип локальности данных).**



Форматы файлов HDFS

- **Parquet,**
- **JSON,**
- **CSV**

… поддерживаются "из коробки", а также есть множество плагинов для различных других форматов данных.

HDFS: Hadoop Distributed File System

- **HDFS - это распределенная файловая система, отказоустойчивая, масштабируемая и чрезвычайно легко расширяемая.**
- **HDFS является основным распределенным хранилищем для приложений Hadoop.**
- **HDFS предоставляет интерфейсы для приложений, чтобы перемещаться ближе к данным.**
- **HDFS разработана, чтобы «просто работать», однако практические знания помогают в диагностике и улучшениях.**
- **Основная система хранения Hadoop.**
- **Файловая система на основе Java.**

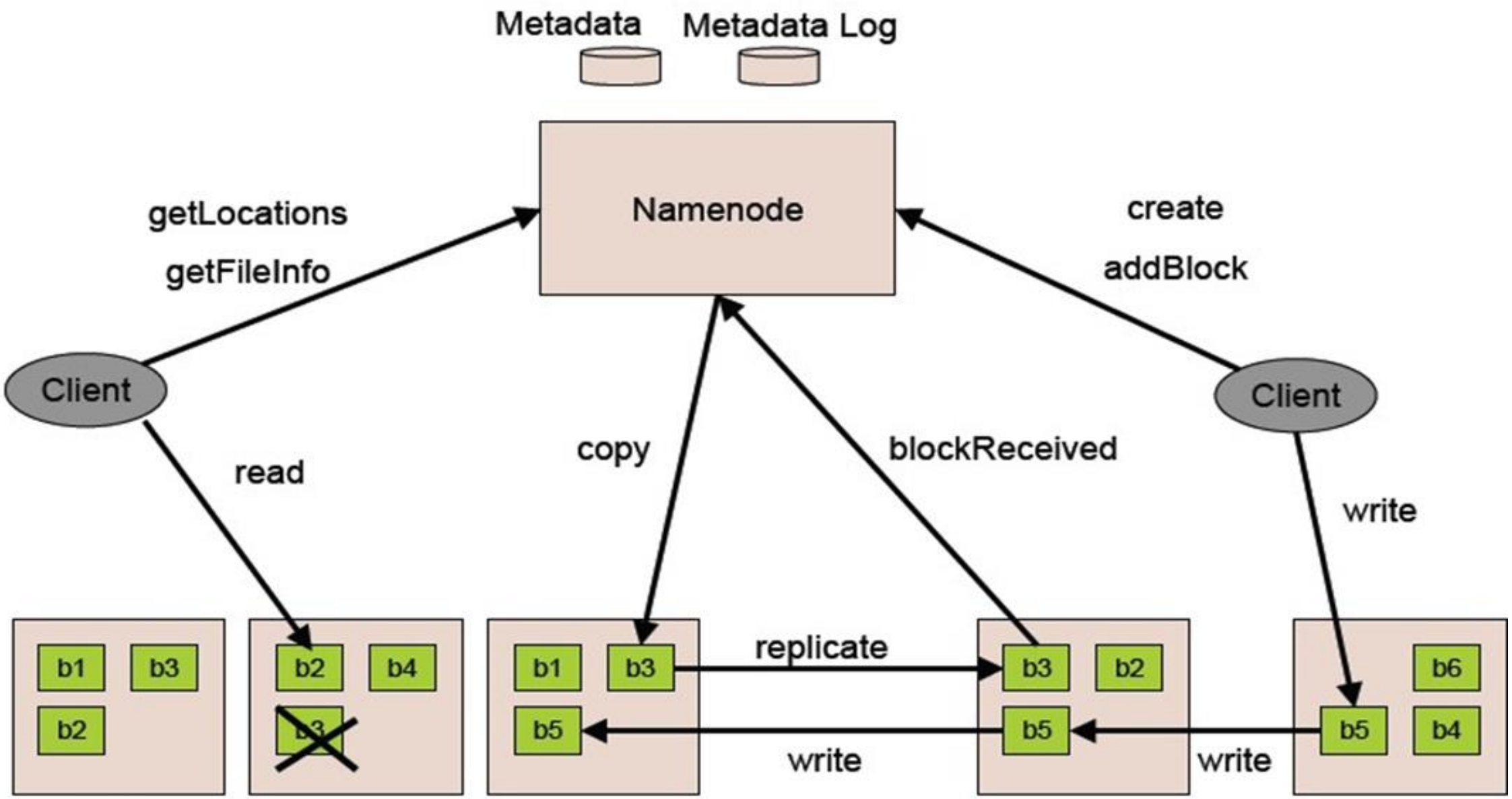
HDFS: особенности

- **Отказоустойчивость - данные дублируются на нескольких узлах данных для защиты от сбоев компьютера. По умолчанию коэффициент репликации равен 3 (каждый блок хранится на трех машинах);**
- **Масштабируемость - передача данных происходит непосредственно с узлами данных, поэтому ваша емкость чтения / записи достаточно хорошо масштабируется с количеством узлов данных;**
- **Объём - нужно больше места на диске? Просто добавьте больше узлов данных и перебалансируйте;**
- **Промышленный стандарт - Другие распределенные приложения построены на основе HDFS (HBase, Map-Reduce)**
- **HDFS предназначена для обработки больших наборов данных с семантикой однократная запись-чтение, она не предназначена для доступа с низкой задержкой.**

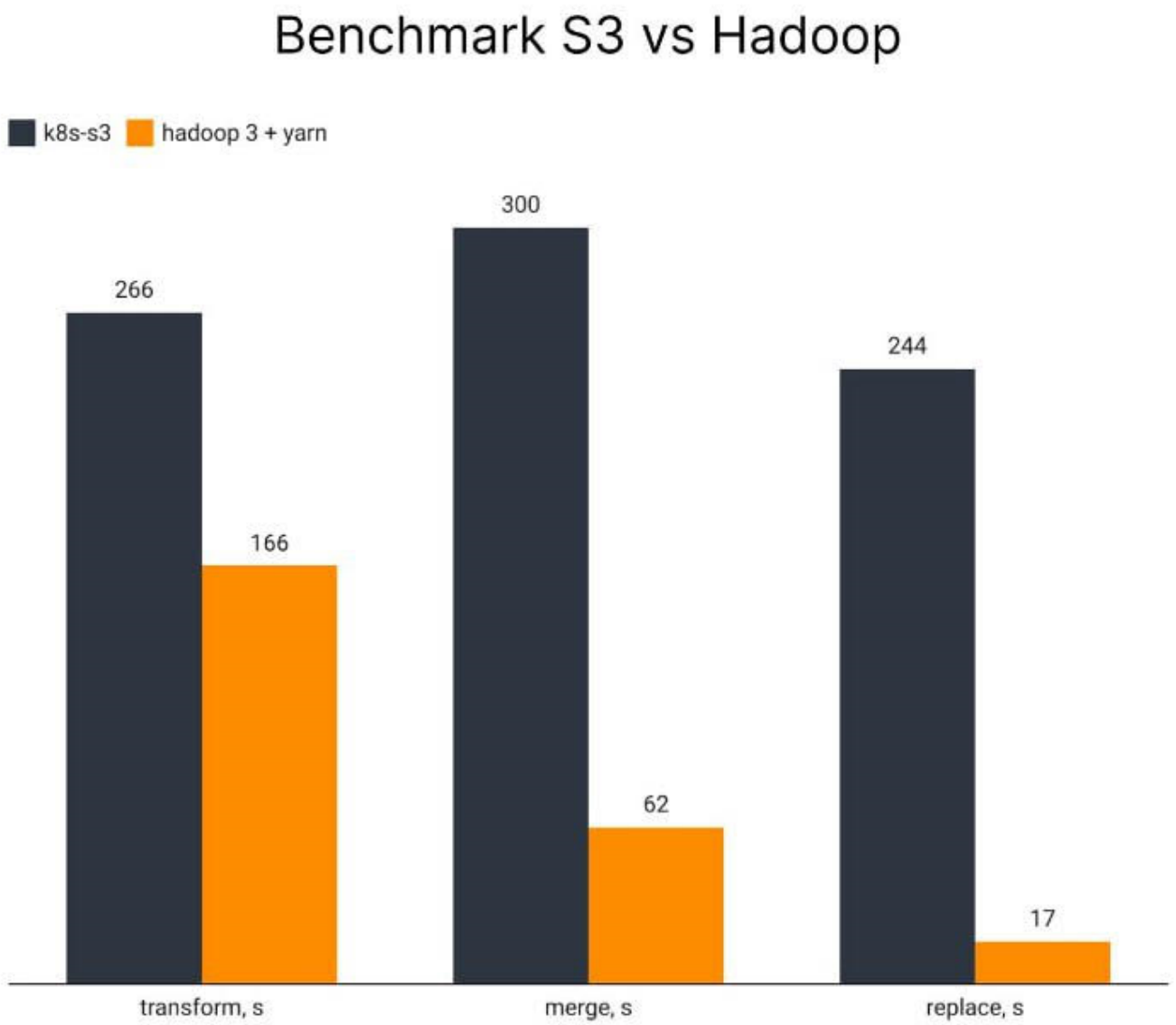
HDFS: хранение данных

- Каждый файл, записанный в HDFS, разбивается на блоки данных
- Каждый блок хранится на одном или нескольких узлах
- Каждая копия блока называется репликой
- Политика размещения блоков
  - Первая реплика размещается на локальном узле
  - Вторая реплика находится в другой стойке
  - Третья реплика находится в той же стойке, что и вторая реплика

HDFS: операции чтения и записи



S3 vs HADOOP


Benchmark S3 vs Hadoop
k8s-s3
hadoop 3 + yarn
266
166
300
62
244
17
transform, s
merge, s
replace, s

Заключение

- **Экосистема Hadoop огромна, и содержит элементы для реализации практически любых задач, связанных с данными;**
- **Hadoop ориентирован на доставку запросов к данным, а не наоборот;**
- **У Hadoop высокое время отклика, поэтому он не подходит для прямого взаимодействия с пользователями.**